

COBRASQ.

Auditoria Técnica do Repositório

COBRASQ - FATURAMENTO

17 de junho de 2026 · Segurança · Bugs · Qualidade · Marca

Auditoria técnica — cobrasq-faturamento

Data: 2026-06-17 · **Branch:** `claude/cobrasq-faturamento-audit-zxqv62` **Escopo:** repositório completo — 3 SPAs (`index.html` , `crm.html` , `calc-juridica.html`), 21 funções serverless em `api/` , 11 Edge Functions Deno em `supabase/functions/` , migrations SQL, banco Supabase `jokbxzhcctcwnbhkhgru` (produção, via advisors read-only), `vercel.json` e ferramental.

Severidades: **P0** crítico · **P1** alto · **P2** médio · **P3** baixo.

Sumário executivo

O backend serverless (`api/`) está **bem arquitetado**: chaves só no servidor, `requireUser` fail-closed, CORS com allowlist real, webhooks com comparação de segredo em tempo constante, idempotência por `UNIQUE` e `regua_envios` com `claim` antes do envio. Os pontos mais sérios estão **no banco em produção** (advisors do Supabase) e em **duas regras financeiras** que podem causar erro de dinheiro. Há também uma **divergência total de identidade visual**: dois dos três apps nunca adotaram o Rebrand Book v3 "Onyx & Ouro".

#	Achado	Sev	Área
1	View <code>profiles</code> expõe <code>auth.users</code> ao <code>anon</code> + é SECURITY DEFINER	P0	Segurança/DB
2	Tabela <code>_backup_cobrasq_data_20260611</code> pública sem RLS	P0	Segurança/DB
3	Repasse PIX duplicável quando <code>repasse_status='preparado'</code>	P1	Bug financeiro
4	Split capital/honorário vira 100% honorário sem acordo vinculado	P1	Bug financeiro
5	<code>portal_tokens</code> com RLS habilitado e sem policy	P1	Segurança/DB
6	Políticas RLS <code>USING(true)</code> (<code>ag_conversations</code> , <code>import_astrea</code>)	P1	Segurança/DB
7	Migrations fragmentadas (<code>/migrations</code> ≠ <code>/supabase/migrations</code>)	P1	Manutenção
8	Identidade visual divergente entre os 3 apps	P1	Marca
9	Proxy <code>api/asaas.js</code> sem allowlist de paths	P2	Segurança
10	<code>api/claude.js</code> sem teto de <code>model / max_tokens</code> (custo)	P2	Segurança/custo
11	12 funções com <code>search_path</code> mutável; senha-vazada off	P2	Segurança/DB
12	Sem CI, lint, <code>package.json</code> ; 1 único teste	P2	Manutenção
13	Idempotência SELECT-then-INSERT (corrida)	P3	Bug
14	Segredo de webhook aceito via <code>?token=</code> (logs)	P3	Segurança
15	UX legada: 88 <code>alert()/confirm()</code> ; helpers de escape divergentes	P3	Qualidade/UX

1. Segurança

1.1 Banco em produção (Supabase advisors)

Coletado via MCP read-only (`get_advisors`). **Não toquei no banco** — correção exige migration, e o `CLAUDE.md` proíbe `db push` cego.

P0 — `auth_users_exposed` + `security_definer_view` A view `public.profiles` expõe dados de `auth.users` ao role `anon` e está definida como `SECURITY DEFINER` , rodando com permissões do criador e **ignorando RLS**. Combinado, qualquer cliente com a anon key (pública) pode ler dados de usuários. → Recriar como `security_invoker=true` , restringir colunas e `REVOKE` do `anon` . Docs: https://supabase.com/docs/guides/database/database-linter?lint=0002_auth_users_exposed

P0 — `rls_disabled_in_public` : `_backup_cobrasq_data_20260611` Tabela de backup pública **sem RLS** no schema exposto ao PostgREST → leitura total via API. → `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` ou mover para schema não exposto / dropar o backup.

P1 — `portal_tokens` com RLS sem policy RLS ligado e nenhuma policy = ninguém (exceto `service_role`) acessa — pode quebrar o portal, ou indica policy esquecida. Confirmar intenção e criar a policy.

P1 — `rls_policy_always_true` `ag_conversations` (UPDATE USING+CHECK true), `import_astrea` (ALL true), `ag_actions` (UPDATE), `login_attempts` (INSERT). Efetivamente sem RLS para `authenticated`. → Restringir por `auth.uid()` /grupo/tenant.

P2 — `function_search_path_mutable` (12 funções) `fin_*`, `ag_*`, `safe_numeric`, `safe_date`, `fn_cobrasq_data_anti_shrink`. → `ALTER FUNCTION ... SET search_path = public, pg_temp` (mitiga sequestro de schema).

P2 — SECURITY DEFINER executáveis por anon/authenticated ~10 funções `/rpc/` (`arquivar_cliente`, `reativar_cliente`, `portal_*` ...). Algumas são intencionais (portal anônimo), mas `arquivar_cliente` / `reativar_cliente` por `anon` merece revisão. → `REVOKE EXECUTE ... FROM anon` onde não for portal público.

P2 — `auth_leaked_password_protection` desabilitado → Ativar checagem HaveIBeenPwned no painel Auth.

1.2 Proxies serverless

- **P2 — `api/asaas.js`**: repassa `?path=` arbitrário à API Asaas com a chave do escritório. Sem cross-host SSRF (host fixo), mas **qualquer usuário logado** alcança qualquer endpoint da conta (criar/cancelar cobrança, `/transfers`). → Allowlist de paths/métodos por perfil.
- **P2 — `api/claude.js` (`api/claude.js:43`)**: repassa `body` inteiro à Anthropic sem limitar `model` nem `max_tokens`. Usuário logado pode gastar a conta. → Validar `model` contra allowlist e impor teto de `max_tokens`.
- **Positivo:** `auth.js` fail-closed, allowlist CORS sem reflexo arbitrário; `config.js` só expõe URL+ANON; `mfa.js` com hash+salt, rate-limit 1/min e máx. 5 tentativas (`api/mfa.js:101,141`).

1.3 Webhooks Edge

- **Positivo:** comparação de segredo em tempo constante (SHA-256 + xor) em `asaas-webhook`, `zapsign-webhook`, `zapi-webhook` (`asaas-webhook/index.ts:33`).
- **P3:** aceitam o segredo via `?token=` na querystring — pode vaziar em logs de borda/proxy. → Preferir header; se manter query, garantir que logs não persistam a URL completa.

1.4 Segredos

Varredura do working tree e histórico (109 commits): **nenhum** `.env` ou **chave real** commitada. Só referências a `process.env` / `Deno.env` e parsing de PEM. ✓

2. Bugs e quebras

P1 — Repasse PIX duplicável (`api/repassar.js:46-48`) Os guards bloqueiam só `repasse_status` `efetuado` e `nao_aplica`. Quando o transfer volta assíncrono (`preparado`), um segundo disparo (duplo-clique/retry) cria **outro** `/transfers` no Asaas → **repasse em dobro**. → Bloquear também `preparado` / `em_processamento`, ou tornar a operação idempotente por `externalReference=op.id` (chechar transfer existente antes de criar).

P1 — Split sem acordo vira 100% honorário (`api/processar-recebimento.js:70-75`) `capitalRatio = capitalBase/acordoTotal`; se não há acordo vinculado, `acordoTotal=0` → `capitalRatio=0` → `valorCapital=0` → `repasse_status='nao_aplica'`. Pagamentos sem acordo são **classificados 100% como honorário e nunca repassam capital ao credor**. → Definir fallback explícito (ex.: usar `devedor.valor_orig` como total) ou marcar a operação para revisão manual em vez de zerar o capital silenciosamente.

P3 — Idempotência por SELECT-then-INSERT (`api/processar-recebimento.js:42-43,100`) Dois webhooks concorrentes para o mesmo pagamento passam o SELECT e ambos tentam INSERT; o `UNIQUE(asaas_payment_id)` protege o dado, mas o 2º vira 500 (retry barulhento). → `upsert` com `on_conflict` / Prefer: `resolution=ignore-duplicates`.

Positivo — `cron-regua.js`: faz *claim* em `regua_envios` antes de enviar (corrige a corrida documentada nos comentários), segredo `CRON_SECRET` constant-time, timezone tratado (12:00 UTC = 09:00 BRT). Robusto.

`_sms.js` é só esqueleto (não implementado) — confirmar que nenhum fluxo ativo depende dele.

3. Qualidade e manutenção

- P1 — Migrations fragmentadas** `/migrations` (raiz, rastreada, parada desde 09/06) contém **só** o módulo financeiro (`2026-05-08_fin_module.sql` etc), que **não existe** em `/supabase/migrations` (60+ arquivos). Contradiz o "fonte única" do `CLAUDE.md` e confunde o rastreio do que está aplicado. → Consolidar em `supabase/migrations/` (com cuidado pelo estado já-aplicado) e remover a raiz. **Não removi automaticamente** — é a única cópia daquelas migrations.
- P2 — Sem CI/lint/ `package.json` ; cobertura mínima** Só `test/f01_devedorToRow.test.js` (passa). Sem linter, sem CI → regressões silenciosas. → Baseline aplicado nesta branch (ver seção "Correções"): `package.json` , ESLint para `api/` , workflow GitHub Actions rodando teste + lint.
- P2 — Performance do banco (advisors):** 274 lints — 141 *multiple permissive policies*, 47 `auth_rls_initplan` (envolver `auth.uid()` em `(select auth.uid())`), 49 índices não usados, 36 FKs sem índice. → Endereçar em lote numa migration de tuning.
- P3 — Duplicação/consistência client-side:** utilitários de data (`_hojeISO` , `_isoMaisDias`) e paletas CSS redeclarados entre apps; helper de escape com nomes divergentes (`eschtml` no index, `escapeHTML` no crm). → Extrair `assets/shared.js` incremental.

4. Ideias e melhorias (produto/técnicas)

- **Toasts no lugar de `alert()`** (88 ocorrências entre os 2 apps) — já existe um toast no `crm.html` (`textContent` em `t`); padronizar e reusar.
- **Observabilidade:** log estruturado + alerta nos webhooks/cron (hoje `console.warn`); painel de "operações com repasse pendente/preparado" para conciliação financeira.
- **Idempotência de saída:** chave de idempotência nas chamadas Asaas `/transfers` .
- **Retry/backoff** nas integrações externas (Asaas/Z-API/ZapSign) e *dead-letter* para webhooks.
- **CSP completa** (hoje só `frame-ancestors`): caminho é extrair o JS inline para arquivos e usar nonce/hash — viabiliza `script-src` sem `unsafe-inline` .
- **Teste de regressão** para o split capital/honorário e para o guard de repasse (itens P1).

5. Identidade visual da Cobrasq

Token canônico: `assets/brand-tokens.css` — **Rebrand Book v3 "Onyx & Ouro"** (`--brand-onyx #0A0908` , `--brand-ouro #B8924B` , `--brand-creme #F1ECE2` ; fontes Fraunces + Inter Tight + JetBrains Mono). **Só `calc-juridica.html` importa esse arquivo.**

P1 — Divergência total: `index.html` e `crm.html` redeclaram `:root` próprio e nunca adotaram o rebrand:

Token	Canônico	<code>index.html</code>	<code>crm.html</code>
navy/link	<code>#0A0908</code> onyx	<code>#002060</code> azul (<code>index.html:22</code>)	<code>#0A1530</code> (<code>crm.html:31</code>)
gold/accent	<code>#B8924B</code> ouro	<code>#FABE44</code> amarelo (<code>index.html:25</code>)	<code>#C9A961</code> (<code>crm.html:8279</code>)
bg	<code>#F1ECE2</code> creme	<code>#FAFAFA</code> (<code>index.html:28</code>)	—
fontes	Fraunces + Inter Tight	Fraunces + Inter + Playfair (<code>index.html:18</code>)	própria
estados	<code>#B23A3A/#2D7D5A/...</code>	<code>#DC2626/#0E9F6E/...</code>	—

São **três paletas e conjuntos de fontes diferentes**. Re-skinnar `index.html` / `crm.html` para o canônico **não é fix trivial** (muda toda a aparência e é decisão de design) — proposto como trabalho dedicado, não aplicado nesta branch. Caminho sugerido: importar `brand-tokens.css` nos três e remover os `:root` locais de cor, validando tela a tela.

Correções aplicadas nesta branch (triviais e seguras)

Apenas aditivas/baixo risco; o resto acima fica como recomendação para aprovação.

1. **Baseline de qualidade:** `package.json` (script `test` + `lint`), config ESLint para `api/` (Node/CommonJS), workflow `.github/workflows/ci.yml` (roda o teste existente + lint).
2. **Comentário obsoleto** em `api/asaas.js` (dizia manter fallback de header já removido).

Não aplicado (requer sua decisão): re-skin de marca (item 5), guards financeiros P1 (itens 3.1/3.2 — alteram lógica de dinheiro), e migrations de segurança/performance do banco (P0/P1/P2 da seção 1.1 — exigem `db push`, vetado pelo `CLAUDE.md` sem revisão).

Auditoria profunda (passe 2) — 2026-06-17

Segundo passe, line-by-line, das áreas que ficaram rasas no passe 1: calculadora jurídica, endpoints `api/` restantes, edge functions e lógica JS dos SPAs. As severidades abaixo são as **minhas** (ajustadas sobre os achados dos agentes; quando rebaixei, explico). "A validar" = precisa de conferência/teste antes de virar fix.

5.1 Calculadora jurídica (`calc-juridica.html`) — a validar (jurídico)

⚠ Nenhuma destas foi corrigida: várias são **decisão jurídica**, exigem seu/advogado OK + casos de teste.

- **P1 — juros pro-rata usa `/30` fixo** (~linha 1350) em vez dos dias reais do mês (`s.diasMes`): juros divergem em fev/meses de 31 dias. (Provável bug de cálculo.)
- **P1/jurídico — cobrança 8% a.a.** (~1594) incide sobre o valor **já corrigido** (semi-composto), não sobre o original. Simples vs composto é **interpretação** (Súmula 121) — confirmar.
- **P1/jurídico — neutralização de deflação** (trava STJ, ~1336) não respeita o pro-rata do segmento. Confirmar regra desejada.
- **P2 — acúmulo de arredondamento** na cobrança (~1592-1598); base da multa não cobre o 3º componente (~1361); honorários podem **duplicar correção** se `atualizar=true` sem data própria (~1479).
- **P3 — `Math.round` em `diffDias`** (linha 911) pode causar off-by-one; floats de correção não arredondados (~1342, só estético).
- **Caso de teste sugerido:** período 01/01/2024 → 28/02/2024 — juros de fev devem diferir dos de jan.

5.2 Endpoints `api/` restantes

- **P1 — `repasse-concluido.js` (TOCTOU):** dois webhooks do mesmo `transferId` podem ler `repasse_status≠'efetuado'` e um `FAILED` sobrescrever um `DONE` já concluído → repasse refeito/duplicado. Fix: `PATCH ... WHERE repasse_status = <esperado>` (update condicional) ou claim por `transferId`.
- **P1/P2 — idempotência sem lock** em `emitir-nf.js` e `emitir-acordo.js` (SELECT-then-act): chamadas concorrentes podem emitir NF/boletos em dobro. Fix: claim idempotente antes de emitir (padrão do `cron-regua.js`).
- **P3 — `mfa.js:145` usa `===`** para comparar hashes (idealmente `crypto.timingSafeEqual`). Rebaixado de P1: compara o `hash` SHA-256 e há teto de 5 tentativas. Melhoria trivial.
- **P2 — `mfa rate-limit` é por `dev_id`** (1 código/min), sem limite por IP nem teto de códigos/hora → brute-force lento ainda possível. Recomenda limite por IP + janela.
- **DISMISSED (era "P0" do agente) — segredo `EMIT_ACORDO_SECRET` compartilhado:** é **intencional** (server-to-server interno; `processar-recebimento` chama `emitir-nf` com esse header). Não é vulnerabilidade.
- **P2 — `diagnostico-financeiro` / `cron-regua` usam `Math.abs` em somatórios:** mascara sinal invertido em dados legados. Validar `tipo_movimento` na origem.
- **P3 — `zapi.js` / `zapsign.js` repassam `?path=`** sem allowlist (mesma classe do Asaas, porém presos ao host e sem endpoint de dinheiro). Recomenda validar o path.

5.3 Edge functions

- **P2 — prompt injection** em `peticao-assistente` e `beatriz-msg`: `contexto` / `contexto_extra` do usuário concatenado no system prompt sem limite. Impacto moderado (texto p/ staff). Fix: truncar (~1000 chars) e estruturar.
- **P2 — `gerar-acordo-termo` casa signatários por índice:** se o ZapSign devolver os signatários em ordem diferente, um devedor recebe o link de assinatura de outro. Fix: casar por CPF/nome. (A validar.)
- **P2 — `cron-mensagens-agendadas` normaliza telefone diferente de `enviar-whatsapp`** (não testa o 9º dígito) → mensagens podem não chegar. Fix: usar a mesma função.
- **P3 — `gerar-acordo-termo` renderiza HTML do termo sem escape** (o HTML vem do app; risco só com usuário logado malicioso); falta de retry/timeout consistente; parsing frágil do JSON da IA em `peticao-assistente`.
- **DOWNGRADE — `asaas-webhook` marcar `cobranca` paga por `externalReference`:** o webhook é autenticado por segredo (só o Asaas chama), então o "UUID roubado" é improvável. Ainda assim, cross-check `cobranca.devedor_id == devedor.id` é boa prática (P3).

5.4 Lógica JS dos SPAs

- **✓ APLICADO — P1 XSS no `showToast` (`index.html:4477`)**: a função injetava a mensagem via `innerHTML` sem escape e ~20 chamadas passam `error.message` do servidor. Corrigido envolvendo a mensagem com `escHtml(msg)` — cobre todos os call sites de uma vez.
- **P1 — `crm.html parseValor()` (~5570)**: o regex remove o ponto de milhar e quebra com número em formato US (`"1,234.56"` → 1). O `index.html` já tem o `parseValorBR()` correto. Fix: reusar essa função. (Dinheiro — não apliquei.)
- **P3 — `crm.html arredondaParaCima()=Math.ceil(v)`** arredonda para **real inteiro**, não centavo; usado em parcelas (~5598). Pode ser "real cheio" intencional — **validar intenção** antes de mexer.
- **P2/P3 — XSS menores: `id` interpolado em `onclick` inline (`index.html:5989` ; id é UUID do banco, baixo risco) e `.nome` de importação CSV em `showToast` (~5374, já mitigado pelo fix do `showToast`).**
- **✓ OK** — autorização é server-side (não confia em `localStorage` para papel/role); dual-write devedor protegido (F-01/F-08); datas via `toISOString` ok.

Correções aplicadas neste passe

1. XSS do `showToast` (`index.html`) — `escHtml(msg)`.

Pendências que exigem sua decisão

- **Calculadora jurídica (5.1)** — confirmar regras (simples×composto, base, trava STJ) + casos de teste.
- **Race conditions financeiras (5.2)** — repasse-concluído / emitir-nf / emitir-acordo: aplico claim idempotente?
- **`parseValor` do CRM (5.2/5.4)** — reusar `parseValorBR` ? (muda parsing de dinheiro)
- **Prompt injection / signatários por índice (5.3)** — aplico os hardenings?